



**REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE TELECOMUNICACIONES E
INFORMATICA (UNETI)
PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA
Ingeniería del Software**

Desarrollo de Aplicación Móvil con Ionic y Angular

Docente:

Carlos Márquez.

Autor:

Vladimir Alejandro Rodríguez Hernández
C.I.: 29.915.065

Valencia, mayo 2026

1. Introducción

Esta evaluación consiste en el desarrollo de una aplicación móvil utilizando el framework Ionic junto con Angular. La aplicación cuenta con tres menús principales: Inicio, Información Personal y Contacto. El objetivo es demostrar el dominio de los conceptos fundamentales de desarrollo móvil híbrido, incluyendo componentes standalone, servicios compartidos, navegación mediante routing y formularios reactivos.

2. Tecnologías Utilizadas

Para el desarrollo de esta aplicación se utilizaron las siguientes tecnologías:

- Ionic Framework: Framework de código abierto para desarrollar aplicaciones móviles y web usando tecnologías web (HTML, CSS, TypeScript).
- Angular (Standalone): Framework de Google para construir aplicaciones web. Se usó la arquitectura standalone, que no requiere NgModules.
- TypeScript: Lenguaje de programación basado en JavaScript que agrega tipado estático.
- Node.js y npm: Entorno de ejecución y gestor de paquetes necesario para instalar y ejecutar el proyecto.
- VS Code: Editor de código utilizado para desarrollar el proyecto.

3. Proceso de Instalación

3.1 Instalación del CLI de Ionic

El primer paso fue instalar el CLI (Command Line Interface) de Ionic de forma global mediante npm:

```
npm install -g @ionic/cli
```

Este comando descarga e instala todas las herramientas necesarias para crear y gestionar proyectos Ionic desde la línea de comandos.

3.2 Creación del Proyecto

Se creó un nuevo proyecto usando la plantilla 'sidemenu', que incluye una estructura base con menú lateral:

```
ionic start miApp sidemenu --type=angular
```

Durante la creación, se seleccionó la opción 'Standalone' para usar la arquitectura moderna de Angular sin NgModules. Esta opción simplifica la estructura del proyecto al no requerir un módulo central.

3.3 Generación de Páginas

Se generaron las tres páginas requeridas usando el CLI de Ionic dentro de la carpeta del proyecto:

```
cd miApp
ionic generate page pages/inicio
ionic generate page pages/informacion
ionic generate page pages/contacto
```

Cada comando generó automáticamente cuatro archivos por página: el componente TypeScript (.ts), la plantilla HTML (.html), los estilos SCSS (.scss) y el archivo de pruebas (.spec.ts).

4. Estructura del Proyecto

La estructura principal del proyecto quedó organizada de la siguiente manera:

```
src/
  app/
    pages/
      inicio/      → Página de bienvenida y registro
      informacion/ → Página de perfil del estudiante
      contacto/    → Página de formulario de contacto
    services/
      estudiante.ts → Servicio compartido de datos
    app.component.html → Menú lateral
    app.component.ts  → Lógica del menú
    app.routes.ts     → Rutas de navegación
```

5. Descripción de los Archivos Principales

5.1 app.routes.ts — Rutas de Navegación

Este archivo define las rutas de la aplicación. Cada ruta conecta una URL con su página correspondiente. Se usa `loadComponent` para cargar cada página de forma lazy (solo cuando se necesita), lo que mejora el rendimiento:

```
export const routes: Routes = [
  { path: '', redirectTo: 'inicio', pathMatch: 'full' },
  { path: 'inicio', loadComponent: () =>
    import('./pages/inicio/inicio.page').then(m => m.InicioPage) },
  { path: 'informacion', loadComponent: () =>
    import('./pages/informacion/informacion.page').then(m => m.InformacionPage) },
  { path: 'contacto', loadComponent: () =>
    import('./pages/contacto/contacto.page').then(m => m.ContactoPage) }
];
```

5.2 services/estudiante.ts — Servicio Compartido

Este servicio actúa como un almacén central de datos del estudiante. Al estar decorado con `@Injectable({ providedIn: 'root' })`, Angular crea una sola instancia compartida entre todas las páginas. Esto permite que los datos ingresados en la página de Inicio sean leídos automáticamente en la página de Información Personal:

```
@Injectable({ providedIn: 'root' })
export class EstudianteService {
  datos = { nombre: '', carrera: '', correo: '', telefono: '' };
  guardar(nombre, carrera, correo, telefono) { ... }
  estaRegistrado(): boolean { return this.datos.nombre.trim() !== ''; }
```

```
| }
```

5.3 app.component.ts — Componente Raíz

El componente raíz controla el menú lateral. Inyecta tanto el MenuController (para abrir y cerrar el menú) como el EstudianteService (para mostrar la inicial del nombre del usuario en el menú). El servicio se declara como público para poder usarlo directamente en el HTML:

```
export class AppComponent {  
  constructor(  
    private menu: MenuController,  
    public estudianteService: EstudianteService  
  ) {}  
  cerrarMenu() { this.menu.close(); }  
}
```

5.4 app.component.html — Menú Lateral

El menú lateral usa ion-menu con type='overlay', lo que significa que se superpone sobre el contenido al abrirse. Dentro del menú se muestra un círculo con la inicial del nombre del usuario registrado (o un ícono si no está registrado), usando *ngIf para mostrar condicionalmente cada opción. Cada ion-item usa routerLink para navegar y llama al método cerrarMenu() al hacer clic:

```
<ion-menu contentId='main-content' type='overlay'>  
  <!-- Avatar con inicial del usuario -->  
  <span *ngIf='estudianteService.estaRegistrado()'>  
    {{ estudianteService.datos.nombre.charAt(0).toUpperCase() }}  
  </span>  
  <!-- Opciones de navegación -->  
  <ion-item routerLink='/inicio' (click)='cerrarMenu()'>Inicio</ion-item>  
</ion-menu>
```

6. Páginas de la Aplicación

6.1 Página de Inicio

La página de inicio cumple dos funciones según el estado del registro del estudiante. Antes del registro, muestra un formulario con cuatro campos (nombre, carrera, correo y teléfono) estilizado con colores navales. Después del registro, muestra una tarjeta con los datos del estudiante.

El componente usa *ngIf para mostrar condicionalmente el formulario o la bienvenida. El método registrar() valida que todos los campos estén completos antes de guardar los datos en el servicio compartido:

```
registrar() {  
  if (!this.nombre || !this.carrera || !this.correo || !this.telefono) {  
    alert('Por favor completa todos los campos.');    return;  
  }  
  this.estudianteService.guardar(this.nombre, this.carrera,  
    this.correo, this.telefono);  
}
```

6.2 Página de Información Personal

Esta página lee los datos directamente del `EstudianteService` y los muestra en una tarjeta visual. No tiene lógica propia ya que solo consume el servicio. Muestra la inicial del nombre en un avatar circular, y debajo los datos de carrera, correo y teléfono con íconos identificativos. Si algún campo está vacío, muestra el texto 'Sin registrar':

```
{{ estudianteService.datos.carrera || 'Sin registrar' }}
```

6.3 Página de Contacto

La página de contacto tiene un formulario con tres campos (nombre, correo y mensaje) que usa two-way data binding mediante `[(ngModel)]` para sincronizar los campos del formulario con las variables del componente en tiempo real.

Al enviar, el método `enviar()` valida que los campos no estén vacíos, agrega el mensaje al arreglo `mensajesEnviados`, y limpia el formulario. Los mensajes enviados se muestran en tarjetas debajo del formulario con el avatar de la inicial del nombre:

```
enviar() {  
  if (!this.nombre || !this.correo || !this.mensaje) {  
    alert('Por favor completa todos los campos.');    return;  
  }  
  this.mensajesEnviados.push({  
    nombre: this.nombre, correo: this.correo, mensaje: this.mensaje  
  });  
  this.nombre = ""; this.correo = ""; this.mensaje = "";  
}
```

7. Conceptos Clave Aplicados

7.1 Componentes Standalone

En lugar de `NgModules`, cada componente declara sus propias dependencias en el array `imports` del decorador `@Component`. Esto significa que si se usa `ion-icon` en el HTML, debe importarse `IonIcon` en el `.ts` del componente. Esto hace cada componente más independiente y fácil de mantener.

7.2 Inyección de Dependencias

Angular crea automáticamente una sola instancia del `EstudianteService` (`providedIn: 'root'`) y la inyecta en cualquier componente que la solicite en el constructor. Esto garantiza que todos los componentes comparten los mismos datos sin necesidad de pasarlos manualmente entre páginas.

7.3 Two-Way Data Binding

El uso de `[(ngModel)]` en los campos del formulario crea un enlace bidireccional: cuando el usuario escribe en el campo, la variable del componente se actualiza automáticamente, y si la variable cambia por código, el campo también se actualiza.

7.4 Directivas *ngIf y *ngFor

La directiva *ngIf permite mostrar u ocultar elementos del HTML según una condición. Se usó para mostrar el formulario antes del registro y la bienvenida después. La directiva *ngFor se usó para iterar sobre el arreglo de mensajes enviados y generar una tarjeta por cada uno.

7.5 Routing con loadComponent

La navegación entre páginas usa lazy loading mediante loadComponent, lo que significa que el código de cada página solo se carga cuando el usuario navega hacia ella. Esto reduce el tiempo de carga inicial de la aplicación.

8. Conclusiones

El desarrollo de esta aplicación permitió aplicar de forma práctica los conceptos fundamentales de Ionic y Angular. Se comprendió la importancia de la arquitectura standalone para simplificar el código, el uso de servicios compartidos para comunicar datos entre páginas, y el sistema de routing para la navegación.

La integración del EstudianteService como fuente única de datos demostró el poder de la inyección de dependencias en Angular, permitiendo que los datos ingresados en una página estén disponibles automáticamente en otras sin necesidad de almacenamiento externo.

El resultado es una aplicación funcional con diseño profesional de tonos navales, menú lateral, formularios con validación y visualización de datos en tiempo real.

Vladimir Rodríguez

C.I. 29.915.065